

Tr	1월	진행 과정 구체화	중요한 변경점/채택점	사용모델A	사용모델B	RSMLE
전처리		데이터 확인 후 price는오른쪽으로 skewed 된 분포인걸 확인 , price == 0 인 행은 제거, train_id 식별자도 제거, 동일행 제거				
전처리		brand_name, category_name, item_description 컬럼들이 결측치가 있는것을 확인 후 그것들을 Other_Null로 fillna를 하여서 단순 삭제 대신 브랜드나 설명이 없는 제품들의 가격은 더 떨어질 확률을 만들어 놓 자리를 만들				
전처리		split_cat(category_name)을 사용하여서 카테고리 Men/Tops/T-shirts런거를 분류하고 분류가 애초 없는 경우 이것도 Other_Null처리하는 방식으로 함. 이런식으로 Other_Null처리 된 컬럼들은 OneHotEncoder/TfidfVectorizer에 넣을때 "결측"이라는 값 자체가 하나의 카테고리/단어처럼 들어가게 됨 -> 즉 Other_Null이라는 자체 sparse matrix 가 따로 있는건 아님				
전처리		preprocessing을 함수를 만들어서 item_description에 토큰한것도 TfidfVectorizer에 넣기 전 텍스트를 정제함 단계일뿐, 토큰화 자체는 matrix가 아니라 컬럼 값을 리스트로 바꾸어 놓음 -> 이걸 desc_vec.fit.trnsform() 에 넣어야 sparse matrix(X_descp).				
전처리		text = re.sub(r"[a-z\s]", '', text) 이 함수는 처리 안함 -> 영문 알파벳과 공백 이외의 것을 전부 삭제하면 iphone 14 pro 256gb가 그냥 iphone 2한 정보를 삭제하는것 (숫자는 보존하는 text_cleaning함수로 바꿈)				
feature engineering의 중요성		Feature selection이 기본적으로 price prediction model 에 있어서 중요한 역할을 하는데 원래 있던 정보/컬럼 들의 combination들이 특히 우리가 원래 잡지 못했던 숨어있는 패턴들을 캐치할 수 있게 된다. 지금 이 Mercari 경우에도 feature들을 6개정도 더 늘리는 방향으로 가져왔던 것도 그러한 이유.				
hstack 처리		hstack에 들어갈것들은 name -> CountVectorizer 되서 , item_description은 토큰화되서 -> TfidfVectorizer, X_cat[brand_name], [cat_dae] 들은 OneHotEncoder, item_condition_id, shipping은 csr_matrix로 들어가서 X_features라는 변수가 만들어진다 / 총 7개의 sparse matrix 가 되는거임, 그리고 이것들이 X_features 안에 녹아 들어가 있는 형태	X_features(전체 feature sparse matrix) + y_labels(log1p 적용된 타겟)			
hstack 처리		Feature engineering이 각 feature block의 형태와 정보를 결정하고, hstack이 그것들을 하나의 X_features로 결합한다. 이후 X_features가 train/test로 분할되고, 그중 X_train이 model.fit()에 직접 들어가기 때문에 feature engineering 결과, 즉 hstack이 모델 학습에 직접적인 영향을 준다.				
6개의 파생 피처 -> feature combination/ feature engineering		X_features에 6개 파생 피처를 추가로 결합하는데 name_len_words, name_len_chars (상품명 단어수), desc_len_tokens (설명 토큰 개수), -> has_brand(브랜드 존재 여부), brand_te(브랜드별 평균 log1p target encoding), cat_so_te (소분류 별 평균 log1p target encoding)				
X_features (hstsack) 구성요소		name CountVectorizer description TF-IDF brand OneHot category OneHot				
feature combination		각 원본데이터에서 새로운 특성을 추출한 다음 -> 이 모든것들을 X_features로 결합함. name_len_words, name_len_chars, desc_len_tokens 는 text structure의 관점으로 바로볼 수 있는데 상품명의 상세도/길이/정보량/ 설명의 상세도 가 들어간다. has_brand, metadata signal, Target_based statistical signal, category_based statistical signal				
구성 요소 파악 -> 모델 예측		약 5만개지만 거의 0인 형태라서 high dimensional + sparse형태를 가짐 -> FM_FTRL (Factorization Machine combined with Follow-the-Regularized-Leader) model과 잘맞음 -interaction algorithm				
train_test_split(80/20)		train_test_split 을 model_train_predict함수 안에서 수행 -> 여기서 80/20 split할때 20%가 X_test와 y_test로 빠지지만 여기서 leakage로 중요하게 model.fit() 기준으로는 맞지만 , 전처리 기준으로는 X_test는 fit된걸로 처리됨. X_test = 테스트 상품의 feature y_test = 그 상품의 실제 가격 정답				
전처리		text = re.sub(r"[a-z\s]", '', text) 이 함수는 처리 안함 -> 영문 알파벳과 공백 이외의 것을 전부 삭제하면 iphone 14 pro 256gb가 그냥 iphone p				
overall approach		LGBM은 원본 sparse 표현 0.49220 -> native categorical + n-gram 0.47363으로 feature representation만 바꿨는데 크게 좋아짐을 확인 -> hstack 처리에 더 힘을 싣려고함->모델 하나 정해서 hyperparameter를 극한으로 튜닝보다는 좋은 sparse X_features를 만든 다음 Ridge와 LGBM 성능 확인하는 절차를 새로 고려함 -> LGBM은 조건을 쪼개면서 feature interaction을 찾기가 수월 -> Apple X Phone처럼 조건과 조건사이의 관계를 캐치할 수 있음.				
n_gram 개선		name / description n-gram 개선 N-gram을 늘리면 무조건 RMSLE가 좋아지지는거 아니기 때문에 경우의 수를 돌려봐서 제일 좋은걸 채택 예정 하지만 의미있는 조합을 찾으려고 피처를 너무 높이논건 배제 그리고 이런 overfitting의 가능성				
max_features 조정		vocab 크기 max_features=20,000 / 30,000 는 사실 불합리적이라 판단. Mercari 처럼 약 150만개 데이터에서 5만개의 text vocab만 쓰는 건 너무 적음/ 모델 별 피처 엔지니어링: 모델마다 자기에게 유리한 인코딩을 그대로 사용 - 불렌딩은 예측값 레벨에서만 결합				
importance of feature interaction		FM_FTRL을 사용함으로써 sparse feature 간 interaction을 잡는다. 이거 사용할때 hstack안에 좋은 구성을 가져감으로서 FM_FTRL을 더 잘활용할 수 있다 (FM 학습방법 예시 Apple x iPhone , iPhone x 128GB)				
feature interaction - lightgbm vs fm_ftrl		LGBM: tree split으로 conditional interaction을 잡음. 하지만 FM_FTRML은 더 복잡한 sparse interaction에 강함 -> TF-IDF / One-hot 같은 수십만~ 백만 차원에 잘맞는다 -> 그래서 둘이 compensate해서 캐글에서도 같은 FM_FTRML 방식을 사용해서 높은 점수를 받을 수 있었던 것				
sparse matrix의 "row index"를 train/valid/test		전체 데이터에서 20% 는 마지막에 최종 성능 확인에 쓰이고 나머지 80%는 1)train 2)valid로 나뉨. valid는 모델을 고르는데 사용하는 데이터. 예를들어서 LGBM의 num_leaves 몇개, learing_rate 얼마? FM_FTRL epcn 몇번, Blend (두개의 LGBM 0.7 / FM 0.3) 을 결정할때 valid가 쓰임 => 그래서 valid로 test하면 안되서 stage 1/2로 나눠어서 test 데이터를 모델 개발과정에서 절대 보지않게 데이터 아키텍처를 짜는 sturcture -> leakage-free				
		LGBM를 early stopping + 정규화(reg_alpha/lambda, feature/bagging_fraction)로 제대로 튜닝해서 0.4732 -> 0.4274로 크게 개선했다.				

[illegible]

[illegible]

[illegible]